Giving coding agents the right knowledge

An architectural recommendation, based on about 1,650 controlled runs of a coding agent

OKF Experiment · October 2026

1. Summary

Coding agents have to understand code before they can change it. When the code is in front of them, they can search it. When a change depends on another team's service, a private library or any code the agent cannot read, the agent has to be given the facts, or it will guess. This paper recommends how to supply those facts.

> **The principle.** Coding agents do not necessarily need more documentation. They need the right information, at the right time, from a source that can be trusted. Where the source of a dependency is out of reach, give the agent an authoritative, versioned contract first. Add structural knowledge only for tasks the contract cannot support. Check, before the agent uses it, that what you supplied matches the version of the dependency in use.

We tested this with one agent on a real Java backend and web storefront. We compared giving the agent nothing, an API contract, a three-kilobyte hand-written note, a generated "guide" of the backend, and the backend's real source. The guide is a folder of markdown files produced from the code by a script, in a format called OKF (Open Knowledge Format). It lists classes, routes and calls, with the file and line each fact came from.

Five results matter for architecture:

1. If the agent can already read the code, extra generated knowledge did not make it more accurate. It cost more tokens, and when it was out of date it misled the agent. That held even for a large repository set up with a guide, a code graph, operating skills and instructions: no reliable gain, and about 48% more tokens.
2. If the agent cannot read a dependency, supplying facts about it made a very large difference: changes passed in about 3 of 20 runs with nothing and in 18 or 19 of 20 with a guide.
3. For changes that depend on an interface (names, paths, fields), a contract or a short note did about as well as the guide, and cost less.
4. For questions about what calls what inside the dependency, only the guide or the source worked. That shows the guide helps agents answer structural questions. It does not show that it helps them make better changes. Tests in a large repository the agent could read found no gain in the quality of its changes (sections 4.5 and 4.6).
5. Out-of-date knowledge produced confident, wrong answers, and nothing placed inside the knowledge itself protected the agent.

2. The problem

Most real changes cross a boundary. A storefront team calls a backend owned by someone else. A service uses a vendored library. A migration touches a system whose source is held in another repository, or is not available to the agent at all. In each case the agent has two options: to be told the facts, or to guess.

Guessing is not harmless. In our tests, agents given nothing guessed the names of request fields from habit and mostly got them wrong. They also spent a great deal of effort searching and then gave up, using about 785,000 tokens per question on average and answering correctly in 1 of 24 cases.

Architects therefore need an answer to a practical question: *what is the cheapest dependable way to give an agent knowledge of code it cannot read?* Generated knowledge bases are one answer. API contracts and short reference notes are others. Until now there was little controlled evidence about how they compare.

3. What we tested

We ran about 1,650 runs of one agent (OpenAI's Codex, model `gpt-5.6-luna`, medium reasoning). Each run was one question or one change, started from scratch, and graded by a hidden test or an exact answer check. We recorded correctness and tokens used, which is a stand-in for cost and time. We called a difference real only when a standard statistical test (Fisher's exact test) gave p below 0.05.

Experiment	Setting	Runs
A, B, C	A small Python library and an invoicing service; the agent can read the code. Which guide shapes help; what an out-of-date guide does.	724
D, E	A web storefront and a 1,173-file Java backend the agent cannot read. Questions; guides with deliberately wrong entries.	336
F, G	The same pair. Small, then harder, cross-repository changes graded by hidden tests.	140
H	Guide against an OpenAPI contract, a note and the source, on changes (H1), call-structure questions (H2) and a small running service (H3).	280
I	A change task in a large codebase the agent can read: does a guide improve the result? Renames with and without a guide.	64
J	The same codebase left as it is, against one set up for agents (guide, code graph, operating skills, `AGENTS.md`): four changes and three structure questions.	112

The backend was Shopizer, with names changed so the agent could not recall the answers from memory. We wrote the questions, the tests and the planted changes ourselves.

4. What we found

4.1 Where the agent can read the code, a guide adds cost, not accuracy

On a small library, agents answered 14 or 15 of 15 questions and made 8 or 9 of 9 changes correctly, with or without a guide. The recommended guide cost 1.30 times the tokens in one run and 1.07 in a rerun (the second not clearly different). Copying in a guide for a library the agent could read cost 1.25

times the tokens, with all 72 changes passing either way.

4.2 Where it cannot, facts decide the outcome

Cross-repository changes passed in 3 of 20 runs with nothing. With a guide they passed in 19 of 20 (small tasks) and 18 of 20 (harder tasks). In the harder tasks, an agent with the backend's real source also passed 18 of 20. The three passes with nothing were on tasks whose answer can be guessed.

4.3 A contract or a note is enough for interface changes

Experiment H gave the same harder tasks to five arms, 16 runs each.

Arm	Passed (of 16)	Tokens per run	Cost against nothing
Nothing	4	277k	baseline
OpenAPI contract	13	192k	reliably cheaper
3 KB note	14	128k	reliably cheaper
Generated guide	14	282k	no clear difference
Source	14	334k	reliably dearer

All four beat nothing. They did not differ reliably from each other. That is not proof they are equivalent: with 16 runs, gaps under about 30 points cannot be separated. The hardest task, building a checkout request whose fields sit in inherited classes, split them: contract 1 of 4, guide 2 of 4, note 4 of 4, source 4 of 4. Two cautions: we wrote the note knowing the tasks, which favours it, and our contract was generated from the source by a script, not by a running application.

4.4 For internal structure, only a guide or the source answered

Six questions asked "which method handles this request, and which class does it then call?" A contract describes an interface, not what a handler calls inside.

Arm	Correct (of 24)	Tokens per run
Nothing	1	447k
OpenAPI contract	8	464k
3 KB note	0	476k
Generated guide	**21**	**170k**
Source	21	207k

This is a clear result, but it is a result about answering questions. It does not show that a better answer produces a better change. The network was on in this experiment, and 56 of 120 runs cloned the public repository; all 8 contract passes were among them. A separate part (H3, a small running service) was contaminated because agents without the source found it on the disk, so we draw no conclusions from it except one: a contract that listed only one success code led agents to treat the service's other success code as an error.

4.5 In a large repository the agent can read, a guide did not improve the change

Experiment I asked whether structural knowledge improves an engineering outcome, not only an answer. In the 1,173-file backend, with the source readable, the agent had to rename a method and update every caller. The names were common (`findOne`, `saveOrUpdate`, `toLocale`), so a plain text search finds three to ten times more hits than there are real callers. A script graded each result exactly: every true call site renamed and nothing else changed. Agents were also kept from reading anything outside their workspace.

Arm	Passed (of 32)	Tokens per run
Source only	22	267k
Source and generated guide	24	318k

The pass rates did not differ reliably ($p = 0.78$), and the guide cost about 22% more tokens, which was reliably more. Failures were mostly missed calls, or edits to callers of a different class with the same method name, and they happened in both arms. The guide's call information lacked one real caller, and the agents did not follow it into error. This is one repository, one kind of task and one model. It does not say that structural knowledge cannot help a change, only that on this task it did not.

4.6 A repository set up for agents, against one left as it is

Experiment J tested the whole recommended setup together, in the same large backend. The *set up* repository had the generated guide, a code graph with a command for asking who calls what, three short "operating skills" that say how to work in the repository (orient, list every place a change touches, check the diff; nothing about the technology), and an instruction file that points to all of them. The *plain* repository had none of this. Four changes and three structure questions were graded exactly, 8 runs each per arm. One change was hard: renaming a generic method used through 58 files, with 105 places to change, some of them text in a configuration annotation that no compiler would catch.

Part	Plain	Set up	Tokens per run
Changes (of 32)	28	29	407k and 526k
Questions (of 24)	16	19	150k and 234k
Hard 58-file rename (of 8)	4	6	1.05M and 1.28M
All runs			1.48 times, reliably more (interval 1.38 to 1.60)

The agents did use the setup: they ran the graph command and opened the guide in every run, and read the navigation skill in 55 of 56. Correctness did not differ reliably in any part or task. The set-up repository cost about 48% more tokens, because the agent read the instructions, the guide and the graph output and then still searched and read the source. The plain agents were already strong, passing 44 of 56 runs, and all 16 agents on the hard rename found and updated the hidden text references without being told. Two caveats: the set-up arm bundles four ingredients, so this does not say which would help alone, and the hard rename favoured it by two runs of eight, which this size of test cannot tell from chance.

4.7 Out-of-date knowledge is the main hazard

We renamed four things in the backend after the guide was made. Four questions had new answers.

The agent had	Gave the old answer (of 16)
The changed source	0
The out-of-date guide	14
The guide plus "read the source to confirm"	16
The guide plus an expiry date on the changed files	16

None of the 46 wrong answers hedged. When the agent could read the code, one sentence in its instructions, "read the source to confirm before you rely on it", cut repeated old facts from 8 of 12 to 0 of 12. When it could not, nothing placed inside the guide helped: not warnings, not expiry dates, not confidence labels. Labels say how a fact was found, not whether it is still true. With an instruction to disclose guesses, agents did say they were relying on a guess in 6 of 8 wrong answers, but accuracy did not improve.

5. Recommendations

Findings are measured. Recommendations are our judgement, built on them. Where a recommendation goes beyond what we measured, it says so.

R1. Do not give agents generated knowledge of code they can already read.
Evidence: experiments A, B, C, I and J. No reliable accuracy gain, higher cost (about 48% more tokens in J, even with skills and a configured instruction file), and a stale guide misleads. *Instead:* rely on the source and existing tools, and add "read the source to confirm before you rely on it" to the agent's instructions.

R2. For a dependency the agent cannot read, supply a versioned API contract first.
Evidence: H1. A contract matched the guide on interface changes at lower cost. *Why architects care:* most services already have, or can generate, one. It is also the form your teams already review and version. *Caution:* a contract states the success codes it was written with, and agents trust it. Make sure it is complete, including every success and error response.

R3. Add a short, authoritative reference for the facts a contract cannot carry.
Evidence: H1, where a 3 KB note was the cheapest arm and among the best. *Use it for:* conventions, the field names people get wrong, error behaviour and business rules. A note written by the dependency's owners costs little and was the most efficient help we measured. This is a measured result with an important bias: we wrote the note knowing the tasks.

R4. Add structural knowledge only where tasks need internal call relationships.
Evidence: H2, where the guide answered 21 of 24 against 8 for a contract. A generated guide is one way to supply it; the source, where it can be shared, is another. *Boundary of the evidence:* we showed better answers to questions, not better changes. Treat this as a targeted capability for specific tasks, such as impact analysis, not as a default layer.

R5. Put business behaviour and invariants in specifications and tests, not in generated knowledge.

Evidence: a generated guide or contract holds structure, not rules. In H3, rules such as price calculation were found only by agents that read the service. That experiment was contaminated, so this is a recommendation, not a measured result. Behaviour that matters should live where it is verified.

R6. Make freshness a gate, not a hope.

Evidence: B and D2. A stale guide of an unreadable dependency was repeated as fact in 14 to 16 of 16 answers. Warnings, banners, expiry dates and labels did not prevent it. *Recommendation:* every piece of knowledge supplied to an agent should record the version of the thing it describes. Check that version against the dependency in use before the agent runs, in your pipeline. We built and ran two such checks (a guide matches its code; a copied-in guide matches its dependency's files) on small examples. They have not been run in a real pipeline, so treat them as a starting point.

R7. Use confidence labels only to make guesses visible.

Evidence: E, F, G. Labels did not make agents more accurate. With an instruction, they made agents say when a conclusion rested on a guess. A wrong fact labelled as certain was trusted every time, so label only what the source states.

6. A reference approach

Think of the knowledge supplied to an agent as layers, each added only when the task needs it. The owner of the dependency publishes each layer, versioned with the code it describes.

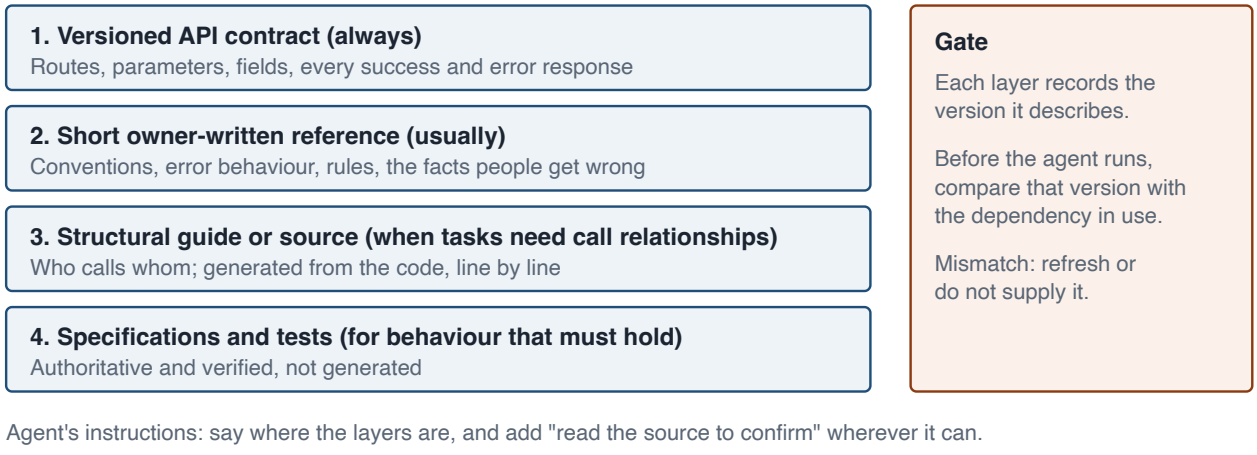


Figure 1. Layers of knowledge, added only as the task needs them, with a freshness check across all of them.

Situation	Suggested approach	Basis
The agent can read the repository	Source and existing tooling, plus "read the source to confirm"	Measured (A, B, C)
It needs another service's API	Versioned API contract	Measured for names and paths (H1)
It needs a few extra facts	Small authoritative reference	Measured, with a note written knowing the tasks (H1)
It needs internal call relationships	Targeted structural knowledge, possibly a generated guide	Measured for questions only (H2)
It needs business behaviour or invariants	Authoritative specifications and tests	Recommendation
Generated knowledge may be stale	Validate against the dependency version before use	Harm measured (B, D2); checks are a recommendation

Operating it

- **Ownership.** The dependency's owner publishes the contract, the reference and, if used, the guide. Consumers copy a pinned version and do not write their own.
- **Pinning and checks.** A copy records the version it came from. A pipeline step compares it with the dependency in use and fails on a mismatch.
- **Instructions.** A short file tells the agent where the layers are, and tells it to read the source to confirm wherever it can.
- **Measure before you scale.** Try your own agent on a handful of real tasks, with and without each layer, before investing in more.

7. What this does not justify

> - It does not justify an enterprise-wide knowledge platform. The experiments compared ways of giving an agent facts about one dependency, not a programme of knowledge management.
> - It does not show a guide is needed everywhere, nor that it improves software changes beyond a contract.
> - It does not show that equal pass rates mean equal quality. Our samples were small.

8. How far to trust this

- **One agent and one setting.** Other models may behave differently.
- **Few tasks.** Experiments A to G had 3 to 5 runs per question per condition; H had 16 to 24 per arm. Small differences are not reliable.
- **We wrote the tests.** The questions, hidden tests, planted changes and the note were ours. The contract was generated by our script.
- **Contamination.** H2 had network access and H3 let agents find material on the disk. We report H2 with and without the affected runs and draw no conclusion from H3.

- **The code was altered.** The backend had names changed and changes applied as uncompiled text edits.
- **Mistakes we made and kept.** Early runs of A and B told the agent to verify in every condition, which hid differences, so we reran them. The first H3 run is archived and unreported. Experiment I's first graded batch was discarded for a grader defect and rerun, and both experiment I and J had ground-truth errors that we found and corrected after the runs (commented-out code counted as required; method names inside configuration strings missed). Neither changed a conclusion. All are published.

9. What to do next

1. **Test a contract and a guide together.** We did not, and it is the combination this paper recommends.
2. **Split the set-up repository.** Test the guide, the graph and the skills one at a time, on tasks the plain agent finds hard, to see whether any one earns its cost.
3. **Test changes that need internal structure.** Show that structural knowledge improves the change, not only the answer.
4. **Repeat on another model and on real drift** from real commits.
5. **Run the freshness checks in a real pipeline** and measure how often they fire.

Evidence, scripts and raw reports: the OKFExperiment repository (README, `docs/DETAILS.md`, `docs/RESULTS.md`, `results/`). Preregistered plans for experiments H and I are in `docs/`.